

Ex. No: 3 System Calls: Fork, Exit, Getpid, Wait, Close

1. fork()

C PROGRAM

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main()
```

```
{
```

```
pid_t pid;
```

```
pid = fork();
```

```
if(pid < 0)
```

```
{
```

```
printf("Fork Failed\n");
```

```
exit(1);
```

```
}
```

```
else if(pid == 0)
```

```
{
```

```
printf("\nCHILD PROCESS");
```

```
printf("\nChild PID : %d", getpid());
```

```
printf("\nParent PID : %d\n", getppid());
```

```
exit(0);
```

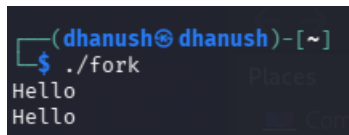
```
}
```

```
else
```

```
{
```

```
wait(NULL);  
printf("\nPARENT PROCESS");  
printf("\nParent PID : %d", getpid());  
printf("\nParent's Parent PID : %d\n", getppid());  
}  
return 0;  
}
```

Output:



```
(dhanush@dhanush)~  
$ ./fork  
Hello  
Hello
```

Shell Script:

```
#!/bin/bash  
echo "Parent Process ID : $$"  
(  
echo "Child Process ID : $$"  
echo "Parent Process ID : $PPID"  
exit 0  
) &  
wait  
echo "Child Process Completed"
```

Output:

```
(dhanush@dhanush)-[~]  
$ bash fork.sh  
Parent Process ID : 49286  
Child Process ID : 49286  
fork.sh: line 9: 25: command not found  
Parent Process ID : 11359  
Child Process Completed
```

2. WAIT()

C PROGRAM

```
#include <stdio.h>
```

```
#include <unistd.h>
```

```
#include <sys/wait.h>
```

```
int main() {
```

```
    pid_t pid;
```

```
    pid = fork();
```

```
    if(pid == 0)    {
```

```
        printf("Child Process Running
```

```
        \n");
```

```
        sleep(5);
```

```
        printf("Child Process Completed
```

```
        \n");    }
```

```
    else{
```

```
        wait(NULL);
```

```
        printf("Parent Resumes Execution
```

```
        \n");
```

```
    }
```

```
    return 0; }
```

Output:

```
(dhanush@dhanush)-[~]
$ gcc wait.c -o wait

(dhanush@dhanush)-[~]
$ ./wait
Child Process Running
Child Process Completed
Parent Resumes Execution

(dhanush@dhanush)-[~]
$
```

SHELL SCRIPT

```
#!/bin/bash (
echo "Child Process Running"
sleep 5
echo "Child Process Completed"
) &
wait
echo "Parent Resumes Execution"
```

Output:

```
(dhanush@dhanush)-[~]
$ bash wait.sh
Child Process Running
Child Process Completed
Parent Resumes Execution
```

3: CLOSE()

C PROGRAM

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
```

```
int fd;

fd = open("sample.txt", O_RDONLY);

if(fd < 0)

{

printf("File Opening Failed\n");

return 1;

}

printf("File Opened Successfully\n");

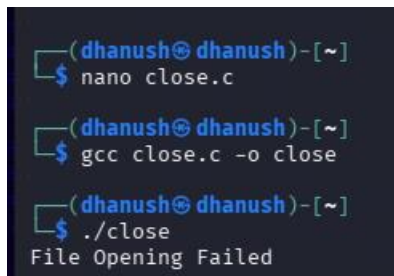
close(fd);

printf("File Closed Successfully\n");

return 0;

}
```

Output:



```
(dhanush@dhanush)~]
$ nano close.c

(dhanush@dhanush)~]
$ gcc close.c -o close

(dhanush@dhanush)~]
$ ./close
File Opening Failed
```

SHELL SCRIPT

Shell scripting does not provide a direct close() system call. File descriptors can be opened and closed

as follows:

```
#!/bin/bash
```

```
exec 3< sample.txt
```

```
echo "File Opened Successfully"
```

```
exec 3<&-
```

```
echo "File Closed Successfully"
```

Output:

```
(dhanush@dhanush)-[~]  
$ bash close.sh  
close.sh: line 3: sample.txt: No such file or directory  
File Opened Ssccessfully  
File Closed Succesfully
```